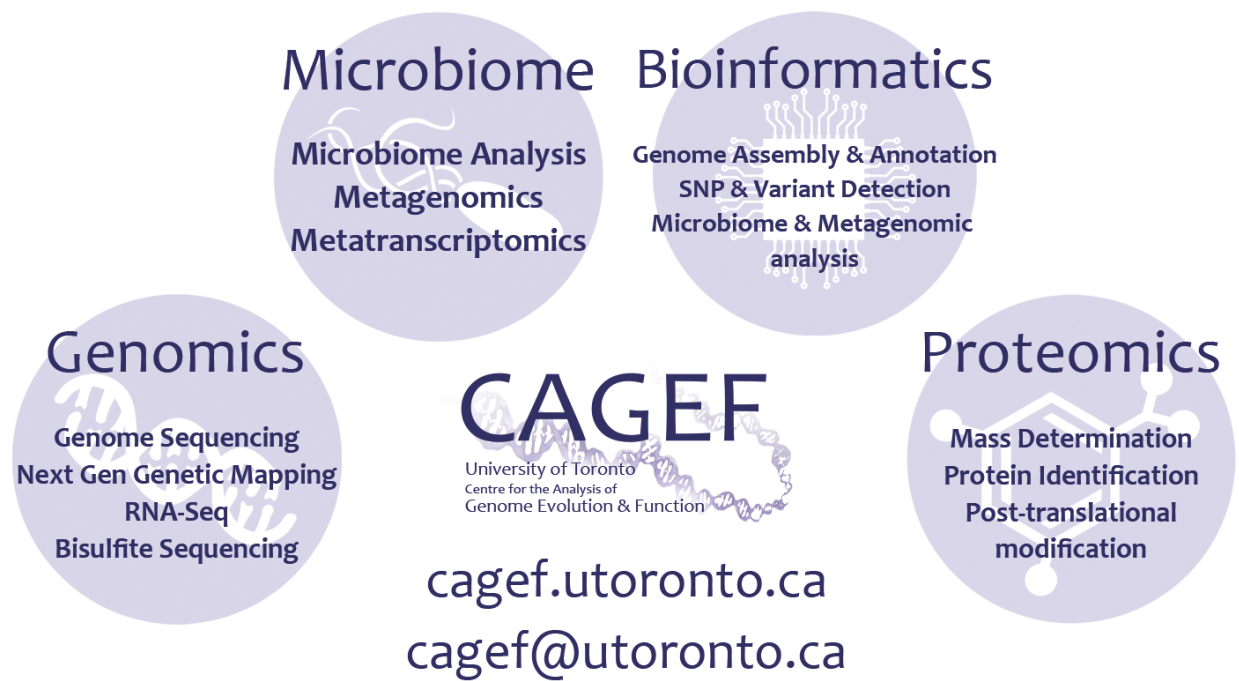




CSB1021HF LEC0131

FUNDAMENTALS OF GENOMIC DATA SCIENCE

0.0.0 Module 4: Introduction to the command line

0.1.0 About Fundamentals of Genomic Data Science

Fundamentals of Genomic Data Science is brought to you by the **Centre for the Analysis of Genome Evolution & Function (CAGEF)** bioinformatics training initiative. This course was developed based on feedback on the needs and interests of the Department of Cell & Systems Biology and the Department of Ecology and Evolutionary Biology.

The structure of this course is a “code-along”, hands-on style! A few hours prior to each lecture, materials will be made available for download at QUERCUS (<https://q.utoronto.ca/>). The teaching materials will consist of a weekly PDF that you can use to follow along with the instructor along with any datasets that you’ll need to complete the module. This learning approach will allow you to spend the time coding and not taking notes!

As we go along, there will be some in-class challenge questions for you to solve. Post lecture assessments will also be available for each module, building upon the concepts learned in class (see syllabus for grading scheme and percentages of the final mark).

0.1.1 Where is this course going?

We’ll take a blank slate approach here to learning genomic data science and assume you know nothing about programming or working directly with next generation sequencing data. From the beginning of this course to the end we want to guide you from potential scenarios like:

- You don’t know what to do with a set of raw sequencing files fresh from a facility like CAGEF.

- You've been handed a legacy pipeline to analyse your data or maintain for the lab, but you don't know what it runs or how.
- You plan on generating high-throughput data but there are no bioinformaticians around to help you out.

and get you to the point where you can:

- Recognize the basic tools in sequence analysis.
- Plan and write your own data analysis pipelines.
- Explain your data analysis methods to labmates, supervisors, and other colleagues.

0.1.2 How do we get there?

In the first half of this course, we'll focus on how to generate analysis pipelines using the Galaxy platform – a user-friendly graphical interface that provides access to common sequence analysis tools. After we are comfortable with these tools, we'll look at life through the lens of a command-line interface. It is here that we will learn the basics of file manipulation and how to program scripts that can carry out multiple tasks for us. From there we'll revisit tools from the first half and learn skills to make your data analysis life easier.

0.2.0 Goals of the module

1. Learn how to move around your file system at the command line.
2. Learn how to manipulate file contents using basic commands.
3. Learn how to combine a series of commands in an executable bash script.
4. Learn how to regulate access and set permissions for files.
5. Learn how to run operations on a remote server through the command line.

0.3.0 Pre-class modules with Coursera

Each week we strongly encourage you to complete the assigned Coursera modules and/or readings **before** class. These are meant to provide you with sufficient background material on each week's module so that we can focus on the act of "doing" something with that data rather than spend a lot of time on the origins of it. You'll find a section outlining the next set of Coursera modules and readings at the end of each module.

0.3.1 Go to www.coursera.org and sign up for an account with your e-mail.

0.3.2 Search the following courses and enroll to audit each course (audit):

- Command Line Tools for Genomic Data Science, Johns Hopkins University.

0.4.0 Setting up your working directory

We suggest that you create a new directory (folder) for this course directly off your root directory called **"FGDS"**. Working from your root directory is not necessary, but it will make some of the aspects of the course a little easier to manage. For Mac OS users, we suggest you create this as a subfolder in your **user** directory.

- 0.4.1 Within this directory, create another directory called **"Module4"**. This is where we will store the data used in this week's module.
- 0.4.2 Create a subdirectory called **"downloads"** to store the initial files as we download them before decompressing and working with them in later steps.

1.0.0 Navigating command-line files

This module is designed as an introduction to some of the most commonly-used commands in the bash shell for genomic data science. Depending on your application, however, there are many shell commands/options that you may find useful. You can find a more detailed summary of some of the most common shell commands in the **section 6.0.0 Appendix**.

Don't be scared to come back to this module to review commands or search new commands online in the future. I've been working at the command line on and off for years and still search and review command options all the time. Most users will not memorize all this stuff! You can also use the `man` command to print the manual for any given command (e.g. `man cd`).

1.0.1 Important command-line wild cards and shortcuts

While working with the command-line there are many tips and tricks we can use to describe or identify files or directories we are searching for. These wildcards are somewhat like regular expressions and let us substitute directly or indirectly for single or multiple filenames.

Command	Description
*	Universal wildcard; anything, any number of times (or nothing)
?	Universal wildcard; anything, once
[a-z]	Any letter or number in range
{a, b}	Any letter or number in list
\	Escape character; ignore special meaning
!	Invert a search

Aside from wildcards, there are also meta-characters or symbols that are used to represent simple ideas or locations when navigating the file system.

Command	Description
.	Current directory
..	Parent directory (one directory above in hierarchy)
~	Root directory (home)

Following these command symbols there are additional keystrokes or characters that allow us to alter commands, how they are run, or stop them completely. Along with this list, we have quick keys and commands that can be used to navigate the terminal commands themselves.

Command	Description
	Pipe output from the left-side command into right-side command
&	Runs command in the background when placed at the end of the command
>	Save the output of a command to a file (e.g. <code>ls -la > allFilesList.txt</code>)
>>	Append the output of a command to a file (e.g. <code>ls -la ../ > allFilesList.txt</code>)
[Tab]	Auto-complete the current command or file path
[Tab+Tab]	List all auto-completed files in your path
[Ctrl+C]	Cancel a process that is currently running
[Ctrl+Z]	Move operation into the background
clear or [Ctrl+L]	Clear all commands and results from terminal window
alias	Set a short form for a command or series of commands
[↑/↓]	Scroll back through the commands you have already entered (aka History)
[Home] or [End] [←/→]	Scroll quickly to the beginning or end of a command, or character by character. Arrows can be combined with [Ctrl] for skipping between space-separated words.

* On Mac OS you may have to substitute the [Ctrl] for the [Cmd] key

1.0.2 Commands for accessing, creating, and removing content

Now that we've reviewed some basic command syntax for identifying files and directory structures, we can look closely at the actual manipulation of directories and files.

Command/Switches	Description
<code>pwd</code>	Print working directory
<code>ls</code>	List files in a directory (e.g. <code>ls -al</code>). Run <code>ls --help</code> to access the manual for this command)
<code>-a</code>	Include hidden files
<code>-l</code>	Show dates and permissions (long listing format).
<code>-1</code>	List all files on separate lines (that's a number 1)
<code>-S</code>	List all files in order of descending size
<code>ll</code>	An aliased equivalent (WSL Ubuntu) to the call <code>ls -alF</code>
<code>cd</code>	Change directory – the primary way of changing your current working directory
<code>mkdir</code>	Make new directory
<code>rmdir</code>	Remove empty directory
<code>rm</code>	Remove file
<code>-r</code>	Recursive removal of all files in directory. This option may also be implemented in many other Unix commands.
<code>-f</code>	While removing files, ignore non-existent files or arguments with prompts (or warnings)
<code>mv</code>	Move a file or a folder to a new location or rename a file
<code>cp</code>	Copy a file to a new location
<code>gzip</code>	Compress files
<code>gunzip</code>	Uncompress (extract) files
<code>tar</code>	Create or expand a tar archive
<code>-c</code>	Create an archive file
<code>-v</code>	Verbosely show the progress
<code>-f</code>	Filename of archive
<code>-r</code>	Append to a specified archive
<code>-x</code>	Extract an archive file
<code>-t</code>	List files in an archive
<code>-z</code>	Filter archive through gzip
<code>passwd</code>	Change your password
<code>history</code>	Access the commands that you have previously used
<code>ctrl + r</code>	Search for a previously run command

1.1.0 Explore your command-line directory with `ls`

Let's practice using some of these commands to navigate our directories in our home directory.

- 1.1.1 Open your command line prompt (Terminal or Bash) and get oriented in your file system using the `pwd` and `ls` commands.

```
pwd
ls
ls -l          # That's a number
ls -l         # That's a letter
ls -la        # Are there hidden files where I am?
```

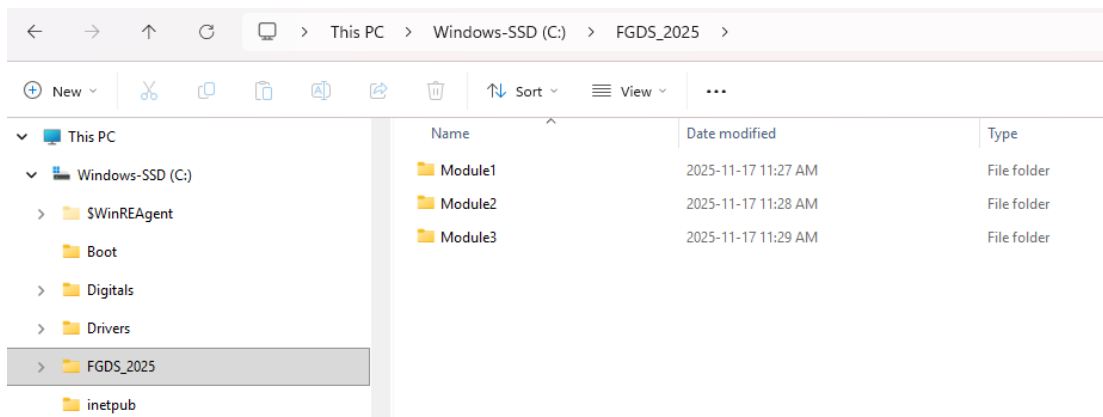
```
mokca@MokData: ~$ pwd
/home/mokca
mokca@MokData:~$ ls
mokca@MokData:~$ ls -l
total 0
mokca@MokData:~$ ls -la
total 32
drwxr-x--- 4 mokca mokca 4096 Nov 15 00:51 .
drwxr-xr-x 3 root root 4096 Nov 14 21:59 ..
-rw----- 1 mokca mokca 21 Nov 14 23:19 .bash_history
-rw-r--r-- 1 mokca mokca 220 Nov 14 21:59 .bash_logout
-rw-r--r-- 1 mokca mokca 3771 Nov 14 21:59 .bashrc
drwx----- 2 mokca mokca 4096 Nov 14 22:00 .cache
drwxr-xr-x 2 mokca mokca 4096 Nov 14 22:00 .landscape
-rw-r--r-- 1 mokca mokca 0 Nov 18 11:13 .motd_shown
-rw-r--r-- 1 mokca mokca 807 Nov 14 21:59 .profile
-rw-r--r-- 1 mokca mokca 0 Nov 15 00:51 .sudo_as_admin_successful
mokca@MokData:~$
```

1.2.0 Change your current directory with `cd`

As we can see, we are currently in our home directory but there isn't much in there. It's likely we haven't moved our FGDS folder into this area yet. For MacOS, you may see plenty of folders – depending on how you've set up your system.

- 1.2.1 Use the Windows Explorer or MacOS Finder to locate your **FGDS** folder which should contain your downloaded data from previous modules. While you're here, make a quick backup copy called **FGDS_backup** and put it somewhere safe (NOT in your original **FGDS** folder)

For instance, mine is located in **C:\FGDS_2025**



The mainstay of moving through directories is the change directory command – `cd`. This will be an important tool in your belt unless you memorize the location of everything in your files.

- 1.2.2 Use the `cd` command to navigate into your FGDS directory using an absolute path.

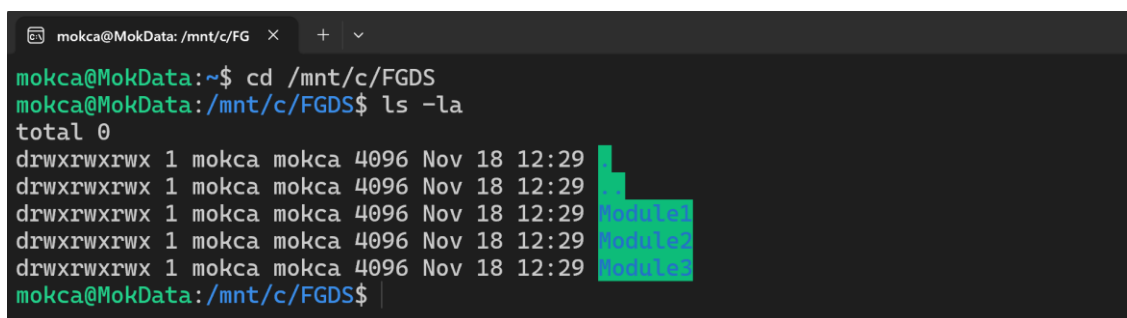
```
cd /mnt/c/FGDS_2025/
```

Note: the location of the FGDS folder will depend on how you've set it up

What is /mnt/? The `/mnt` folder is the portal to the various hard drives on your computer. In many cases you may only have a single drive “mounted” to your system, but others may have additional letter drives representing partitions or USB drives! The command we used above is an absolute path meaning you can reach it from *anywhere* in your own system or set of folders.

- 1.2.3 Take a look around at the directory structure. You should see folders from our previous modules. For the remainder of the examples, we'll assume our data is in the folder "FGDS".

```
ls -la
```

A terminal window with a dark background. The prompt is 'mokca@MokData:~/mnt/c/FG\$'. The user enters 'cd /mnt/c/FGDS' and the prompt changes to 'mokca@MokData:/mnt/c/FGDS\$'. Then the user enters 'ls -la' and the output shows a directory listing with permissions, owner, group, size, date, and time for five files, all named 'Module1'. The files are highlighted in green in the original image.

```
mokca@MokData:~/mnt/c/FG$ cd /mnt/c/FGDS
mokca@MokData:/mnt/c/FGDS$ ls -la
total 0
drwxrwxrwx 1 mokca mokca 4096 Nov 18 12:29 Module1
drwxrwxrwx 1 mokca mokca 4096 Nov 18 12:29 Module1
drwxrwxrwx 1 mokca mokca 4096 Nov 18 12:29 Module1
drwxrwxrwx 1 mokca mokca 4096 Nov 18 12:29 Module1
drwxrwxrwx 1 mokca mokca 4096 Nov 18 12:29 Module1
mokca@MokData:/mnt/c/FGDS$
```

- 1.2.4 From the FGDS directory, navigate directly to the **Module1** directory using a relative path.

```
cd ./Module1      # the "." Means "from the current directory"
```

- 1.2.5 From the **Module1** directory navigate directly to the **Module3** directory using a relative path.

```
cd ../Module3
```

- 1.2.6 Return to the **home** directory.

```
cd ~/              # 'cd ~' will also work
```

1.3.0 Move files and directories with `mv`

Now that we've located our FGDS folder, we can move it to our home directory if it isn't already located there. The `mv` command serves dual purposes, mainly for moving files from one place to another. As we'll see, during the process you must also specify a destination folder. You may, however, also define a destination filePath, essentially renaming the file or directory as you move it! The `mv` command takes on the form of:

```
mv filePathFrom filePathTo
```

Note that for directories, if your `filePathTo` exists, then the `filePathFrom` will be moved into that directory. If `filePathTo` does not exist, then the directory will be renamed as it is moved. Additionally, when moving files, destination files can be overwritten or replaced if `filePathTo` already exists!

- 1.3.1 If your "FGDS" directory is not already in your home directory, move it and all its contents there using the `mv` command.

Recall that if you are working on Windows 10/11, you can access your entire Windows file structure from the "mnt" directory.

- 1.3.2 Move your FGDS directory if necessary

```
cd ~/              # Return to your home directory
mv /mnt/c/FGDS ./
                  # Move the original FGDS folder using the right name!
ls -la
```

```

mokca@MokData:~$ ls -la
total 36
drwxr-x--- 5 mokca mokca 4096 Nov 18 12:33 .
drwxr-xr-x 3 root root 4096 Nov 14 21:59 ..
-rw----- 1 mokca mokca 21 Nov 14 23:19 .bash_history
-rw-r--r-- 1 mokca mokca 220 Nov 14 21:59 .bash_logout
-rw-r--r-- 1 mokca mokca 3771 Nov 14 21:59 .bashrc
drwx----- 2 mokca mokca 4096 Nov 14 22:00 .cache
drwxr-xr-x 2 mokca mokca 4096 Nov 14 22:00 .landscape
-rw-r--r-- 1 mokca mokca 0 Nov 18 11:13 .motd_shown
-rw-r--r-- 1 mokca mokca 807 Nov 14 21:59 .profile
-rw-r--r-- 1 mokca mokca 0 Nov 15 00:51 .sudo_as_admin_successful
drwxrwxrwx 5 mokca mokca 4096 Nov 18 12:29 .vscode
mokca@MokData:~$

```

1.4.0 Create new directories with `mkdir`

- 1.4.1 Create a new directory within **FGDS** for this module called **Module4**. This will require us to use the `ls`, `cd`, and `mkdir` commands, but we will also explore how the `rmdir` command works so that you know how to remove a directory. Notice your **Module4** directory is now available in your Finder/File Explorer

```

cd FGDS
ls
mkdir ModuleFour      # Make a directory
ls
rmdir ModuleFour      # Remove the directory (only if empty)
ls
mkdir Module4
ls -la
cd Module4
ls
pwd                    # print the current working directory

```

```

mokca@MokData:~/FGDS$ ls
Module1  Module2  Module3
mokca@MokData:~/FGDS$ mkdir ModuleFour
mokca@MokData:~/FGDS$ ls
Module1  Module2  Module3  ModuleFour
mokca@MokData:~/FGDS$ rmdir ModuleFour
mokca@MokData:~/FGDS$ ls
Module1  Module2  Module3
mokca@MokData:~/FGDS$ mkdir Module4
mokca@MokData:~/FGDS$ ls -la
total 24
drwxrwxrwx 6 mokca mokca 4096 Nov 18 12:35 .
drwxr-x--- 5 mokca mokca 4096 Nov 18 12:33 ..
drwxrwxrwx 3 mokca mokca 4096 Nov 18 12:29 Module1
drwxrwxrwx 3 mokca mokca 4096 Nov 18 12:29 Module2
drwxrwxrwx 3 mokca mokca 4096 Nov 18 12:29 Module3
drwxr-xr-x 2 mokca mokca 4096 Nov 18 12:35 Module4
mokca@MokData:~/FGDS$ cd Module4
mokca@MokData:~/FGDS/Module4$ ls
mokca@MokData:~/FGDS/Module4$ pwd
/home/mokca/FGDS/Module4
mokca@MokData:~/FGDS/Module4$

```

1.5.0 Copy files with `cp`

You should now be in the **Module4** directory. Copy **HinfKW20_genomic.fna**, **HinfKW20_cds.fna**, and **HinfKW20_protein.faa** files from your **Module1** directory into your **Module4** directory.

In doing so, you will experiment with the `mv`, `cp`, and `rm` commands. We will also explore how to do this all at once with wildcards. Note that both the `.` and `[tab]` meta-characters and commands can make your life a lot easier here by specifying your current location in the filesystem and auto-filling filenames, respectively. You can also scroll up and down previous commands that you want to repeat using the up $\uparrow$ and down $\downarrow$ arrows.

1.5.1 Copy files individually from **Module1** over to **Module4**

```
pwd                # Make sure you are in the Module4 directory!
ls
ls ../Module1
ls ../Module1/downloads
cp ../Module1/downloads/HinfKW20_genomic.fna .
cp ../Module1/downloads/HinfKW20_cds.fna .
cp ../Module1/downloads/HinfKW20_protein.faa .
ls
```

```
mokca@MokData:~/FGDS/Module4$ pwd
/home/mokca/FGDS/Module4
mokca@MokData:~/FGDS/Module4$ ls
mokca@MokData:~/FGDS/Module4$ ls ../Module1
2024_HinfKW20
2024_HinfKW20.sorted.bam
HinfKW20_cds.fna
HinfKW20_features.gtf
mokca@MokData:~/FGDS/Module4$ ls ../Module1/downloads
2024_HinfKW20_genomic.fna
2024_HinfKW20_genomic.fna.fai
HinfKW20_genomic.gbff
HinfKW20_genomic.gff
HinfKW20_protein.faa
mokca@MokData:~/FGDS/Module4$ cp ../Module1/downloads/HinfKW20_genomic.fna .
mokca@MokData:~/FGDS/Module4$ cp ../Module1/downloads/HinfKW20_cds.fna .
mokca@MokData:~/FGDS/Module4$ cp ../Module1/downloads/HinfKW20_protein.faa .
mokca@MokData:~/FGDS/Module4$ ls
2024_HinfKW20_cds.fna 2024_HinfKW20_genomic.fna 2024_HinfKW20_protein.faa
mokca@MokData:~/FGDS/Module4$
```

1.5.2 Remove the **Module4** directory and remake it

```
cd ..                # Return to ~/FGDS/
rmdir Module4        # What happens when we use this command?
rm -r Module4
ls
mkdir Module4
cd Module4
```

```
mokca@MokData:~/FGDS$ rmdir Module4
rmdir: failed to remove 'Module4': Directory not empty
mokca@MokData:~/FGDS$ rm -r Module4
mokca@MokData:~/FGDS$ ls
Module4
mokca@MokData:~/FGDS$ mkdir Module4
mokca@MokData:~/FGDS$ cd Module4
mokca@MokData:~/FGDS/Module4$ ls
mokca@MokData:~/FGDS/Module4$
```

1.5.3 Copy all of the .fna and .faa files over from **Module1**.

```
cp ../Module1/downloads/HinfKW20_*.f[a,n]a .
ls
mv HinfKW20_genomic.fna ../
ls
ls ../
mv ../HinfKW20_genomic.fna .
ls -la
```

```
mokca@MokData:~/FGDS/Module4$ cp ../Module1/downloads/HinfKW20_*.f[a,n]a .
mokca@MokData:~/FGDS/Module4$ ls
HinfKW20_cds.fna  HinfKW20_genomic.fna  HinfKW20_protein.faa
mokca@MokData:~/FGDS/Module4$ mv HinfKW20_genomic.fna ../
mokca@MokData:~/FGDS/Module4$ ls
HinfKW20_cds.fna  HinfKW20_protein.faa
mokca@MokData:~/FGDS/Module4$ ls ../
HinfKW20_genomic.fna  Module1  Module2  Module3  Module4
mokca@MokData:~/FGDS/Module4$ mv ../HinfKW20_genomic.fna .
mokca@MokData:~/FGDS/Module4$ ls -la
total 4304
drwxr-xr-x 2 mokca mokca  4096 Nov 18 12:45 .
drwxrwxrwx 6 mokca mokca  4096 Nov 18 12:45 ..
-rwxr-xr-x 1 mokca mokca 1875338 Nov 18 12:44 HinfKW20_cds.fna
-rwxr-xr-x 1 mokca mokca 1853084 Nov 18 12:44 HinfKW20_genomic.fna
-rwxr-xr-x 1 mokca mokca  665876 Nov 18 12:44 HinfKW20_protein.faa
mokca@MokData:~/FGDS/Module4$
```

1.6.0 Create a compressed archive with tar or gzip

You might recall that through working with our data in **Module1-3**, many of our downloads come in compressed and/or archived files. It's time for us to practice using some of the command-line tools to create and work with these space-saving files. The **tar** (tape archive) command provides a way to create and extract from archive files but does not *directly* compress them. The tar command takes the form of:

```
tar -switches archiveName archiveContents
```

Some useful **tar** command switches:

Switch/flag	Description
-f --file	The file or archive you want to specify
-t --list	List the files present in an archive
-c --create	Create an archive
-x --extract	Rather than build an archive, you can use this tag to <i>extract</i> an archive.
-v --verbose	Display all of the file names in the archive
-z --gzip	Filter the archive through gzip (especially great for tar.gz files). Both for compression and decompression depending on other flags!
-k --keep-old-files	Don't replace existing files when extracting

1.6.1 Create a **tar** archive that contains all the gene, rna, and protein sequence information that you just moved to your **Module4** directory.

```
ll
tar --help
tar -cvf HinfKW20_sequences.tar HinfKW20_*
# So we want to create an archive with the specified file name
# Note how the f is on the end!
ll
```

```

mokca@MokData:~/FGDS/Module4$ tar -cvf HinfKW20_sequences.tar HinfKW20_*
HinfKW20_cds.fna
HinfKW20_genomic.fna
HinfKW20_protein.faa
mokca@MokData:~/FGDS/Module4$ ls -la
total 8604
drwxr-xr-x 2 mokca mokca 4096 Nov 18 12:49 .
drwxrwxrwx 6 mokca mokca 4096 Nov 18 12:45 ..
-rwxr-xr-x 1 mokca mokca 1875338 Nov 18 12:44 HinfKW20_cds.fna
-rwxr-xr-x 1 mokca mokca 1853084 Nov 18 12:44 HinfKW20_genomic.fna
-rwxr-xr-x 1 mokca mokca 665876 Nov 18 12:44 HinfKW20_protein.faa
-rw-r--r-- 1 mokca mokca 4403200 Nov 18 12:49 HinfKW20_sequences.tar
mokca@MokData:~/FGDS/Module4$

```

The **gzip** command has one function – to compress or expand files using the **gzip** algorithm. When we are using **gzip** on multiple files, it will accept them as input but it will individually compress them. By default, it will replace the original file(s) with the gzipped versions (.gz).

- 1.6.2 Create a compressed **gzip** version of **HinfKW20_cds.fna** and **HinfKW20_genomic.fna**. Take a look at the resulting files. Now decompress the same files with **gunzip**.

```

gzip --help
gzip HinfKW20_cds.fna HinfKW20_genomic.fna
ll
gunzip *.gz
ll

```

- 1.6.3 Create a compressed **gzip** version of the tar archive. By archiving before compressing the file, everything is kept together as a group!

```

gzip HinfKW20_sequences.tar
ll

```

```

mokca@MokData:~/FGDS/Module4$ gzip HinfKW20_sequences.tar
mokca@MokData:~/FGDS/Module4$ ll
total 5748
drwxr-xr-x 2 mokca mokca 4096 Nov 18 12:51 ./
drwxrwxrwx 6 mokca mokca 4096 Nov 18 12:45 ../
-rwxr-xr-x 1 mokca mokca 1875338 Nov 18 12:44 HinfKW20_cds.fna*
-rwxr-xr-x 1 mokca mokca 1853084 Nov 18 12:44 HinfKW20_genomic.fna*
-rwxr-xr-x 1 mokca mokca 665876 Nov 18 12:44 HinfKW20_protein.faa*
-rw-r--r-- 1 mokca mokca 1477022 Nov 18 12:49 HinfKW20_sequences.tar.gz
mokca@MokData:~/FGDS/Module4$ gzip -l HinfKW20_sequences.tar.gz
      compressed      uncompressed      ratio uncompressed_name
      1477022          4403200      66.5% HinfKW20_sequences.tar

```

- 1.6.4 Check how much our original archive folder was compressed vs the original version using the **list** switch **-l**.

```

gzip -l HinfKW20_sequences.tar.gz
ls -l

```

1.6.5 Unarchive *and* decompress your **tar.gz** file by using **tar** and filtering it through **gzip**.

```
rm HinfKW20_*.f*          # remove the current files
ll
tar -zxvf HinfKW20_sequences.tar.gz  # x stands for extract!
ls -l
rm HinfKW20_sequences.tar.gz
ll
```

```
mokca@MokData:~/FGDS/Module4$ tar -zxvf HinfKW20_sequences.tar.gz
HinfKW20_cds.fna
HinfKW20_genomic.fna
HinfKW20_protein.faa
mokca@MokData:~/FGDS/Module4$ ls -l
total 5740
-rwxr-xr-x 1 mokca mokca 1875338 Nov 18 12:44 HinfKW20_cds.fna
-rwxr-xr-x 1 mokca mokca 1853084 Nov 18 12:44 HinfKW20_genomic.fna
-rwxr-xr-x 1 mokca mokca 665876 Nov 18 12:44 HinfKW20_protein.faa
-rw-r--r-- 1 mokca mokca 1477022 Nov 18 12:49 HinfKW20_sequences.tar.gz
```

1.6.6 Prior to this we used both **tar** and **gzip** to accomplish the creation of a tar.gz file but much like our last call to **extract** a .gz file, we can create a compressed archive **tar.gz** file by using **tar** and filtering it through **gzip**.

```
tar -czvf HinfKW20_sequences.tar.gz HinfKW20_*.f*
ls -l
```

1.6.7 How do we view the contents of a tar.gz file? We already know how to do it with a .gz file. Let's compare

```
gzip -l HinfKW20_sequences.tar.gz  # Show the compressed contents
tar -tf HinfKW20_sequences.tar.gz  # Show the archive contents
tar -tvf HinfKW20_sequences.tar.gz  # Show archive contents verbosely
rm HinfKW20_sequences.tar.gz
ll
```

```
mokca@MokData:~/FGDS/Module4$ gzip -l HinfKW20_sequences.tar.gz
      compressed      uncompressed  ratio uncompressed_name
      1476999          4403200    66.5% HinfKW20_sequences.tar
mokca@MokData:~/FGDS/Module4$ tar -tf HinfKW20_sequences.tar.gz
HinfKW20_cds.fna
HinfKW20_genomic.fna
HinfKW20_protein.faa
mokca@MokData:~/FGDS/Module4$ tar -tvf HinfKW20_sequences.tar.gz
-rwxr-xr-x mokca/mokca 1875338 2024-11-18 12:44 HinfKW20_cds.fna
-rwxr-xr-x mokca/mokca 1853084 2024-11-18 12:44 HinfKW20_genomic.fna
-rwxr-xr-x mokca/mokca 665876 2024-11-18 12:44 HinfKW20_protein.faa
```

1.7.0 Simplify commands with the `alias` command

By now you will have noticed the copious use of `ls -la` to view the details of all the files both visible and hidden within our directories. In Windows WSL/Ubuntu, this command has already been shortened to the `ll` command for us. For our MacOS users, however, we can set up the same shortcut command using `alias` which takes the form of:

```
alias <newCommand>='<oldCommand>'
```

Note, however, that the alias command is meant to represent evaluated expressions or commands so they are expected to execute some kind of command in the bash shell.

1.7.1 Create an alias for the `ls -la` command if you are running MacOS.

```
alias ll='ls -la'
ll
```

1.7.2 Create an alias to represent the location of the **Module4** folder

```
alias mod4='cd ~/FGDS/Module4'    # check it out with 'cd ~/' and 'mod4'
```

1.7.3 Remove your alias with the `unalias` command

```
unalias mod4
```

1.8.0 Set variables in bash for simplifying paths

As an extension of our work with alias, you may find that sometimes you are working with very long file paths because your data is distributed all around various places. Paths could be a specific file or just a folder where files are stored. Similar to an alias, *variables* can be used to represent specific file paths which can then be *substituted into* your bash commands. Variables take the form of:

```
<variableName>=<pathToFile>
```

To access your variables in the commandline, you must precede them with the `$` sign.

1.8.1 Create a variable to the **Module4** path. Note the lack of spaces in the command!

```
mod4=~ /FGDS/Module4/
```

1.8.2 Check if your variable works!

```
cd ~
```

```
cd $mod4
```

Aliases now and forever? Note that for both aliases and variables, they **only persist during your current Linux session**. When you close the terminal Window, these will disappear and not be available during your next session. However, if you want to make these more permanent, you can update your `.bashrc` (Windows) or `.bash_profile` / `.zshrc` (MacOS) to recreate these with every new session. See section **3.5.0** for more information on altering these files!

2.0.0 Viewing and manipulating file contents

Now that you have a sense of how you can move around your file system to create and remove content, let's look at some important commands for viewing and manipulating file contents. There are different terminal commands that can be used to quickly view, edit, summarize, combine, and search content.

2.1.0 Use the `less` command to view content

The `less` command allows you to view the contents of a file by moving through it both forward and backwards. It comes equipped with hot-keys or commands that make it easy to navigate your text files one at a time.

Keys	Description
[Arrow keys], [space], [page-up], [page-down]	Navigate around the page in a forwards or backwards direction
b	Return to previous page
Line_numg	Jump to a line number, default is the start of the file (e.g. 1000g)
Line_numG	Jump to a line number, default is the end of the file (e.g. 46G)
/	Search for text in the file (e.g. /ribonuc)
n	Find next occurrence in search
N	Find previous occurrence in search
-N	Display the line numbers in your file
-S	Toggle on/off word wrapping in your display
q	Exit the file

- 2.1.1 Copy the **HinfKW20_features.gtf** file from your **Module1** directory into your **Module4** directory with the `cp` command. Then create a copy of this file titled **HinfKW20_features_copy.gtf**

```
cp ../Module1/downloads/HinfKW20_features.gtf .
cp HinfKW20_features.gtf HinfKW20_features_copy.gtf
```

- 2.1.2 Explore the **HinfKW20_features_copy.gtf** file with the `less` command. Jump to line **1000** and then search for the genes **mutS** and **2-isopropylmalate**.

```
less HinfKW20_features_copy.gtf
-N                                # Display line numbers in the file
1000g                            # Go to line 1000
/mutS                            # Look for the mutS gene
/2-isopropylmalate               # Look for 2-isopropylmalate
q                                # quit less
```

```
3984 "HI_0986"; note "similar to GB:D10483 SP:P09151 PID:216492 GB:U00096 PID:1786261 percent identity: 67.19
3984 uct "2-isopropyl
3984 malate synthase (leuA); protein_id "AAC22647.1"; transl_table "11"; exon_number "1";
3985 L42023.1      Genbank start_codon      1043307 1043309 .      +      0      gene_id "HI_0986"; transcr
3985 ocus_tag "HI_0986"; note "similar to GB:D10483 SP:P09151 PID:216492 GB:U00096 PID:1786261 percent identit
3985 e"; product "2-isopropylmalate synthase (leuA); protein_id "AAC22647.1"; transl_table "11"; exon_number
3986 L42023.1      Genbank stop_codon       1044900 1044902 .      +      0      gene_id "HI_0986"; transcr
3986 ocus_tag "HI_0986"; note "similar to GB:D10483 SP:P09151 PID:216492 GB:U00096 PID:1786261 percent identit
3986 e"; product "2-isopropylmalate synthase (leuA); protein_id "AAC22647.1"; transl_table "11"; exon_number
3987 L42023.1      Genbank gene             1044988 1046064 .      +      .      gene_id "HI_0987"; transcript_id
3987 us_tag "HI_0987"; note "HI0987";
```

- 2.1.3 Comprehension challenge: on what lines would you find “galactoside ABC transporter, ATP-binding protein (mgIA)”?

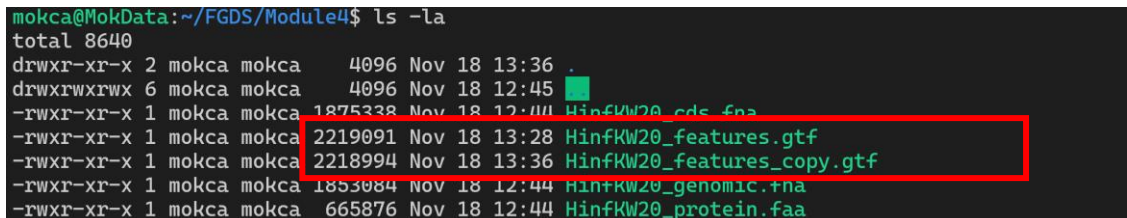
2.2.0 Use `vi` to edit your text files

The text editor `vi` is a standard component of the Linux operating system and was created in 1976. While intimidating at first, this can be a very helpful editor when working with your files.

Key	Description
Arrow keys	Move around the file
i	Editing mode. This is great for preventing accidental changes to your file!
esc	Exit editing mode
:q	Quit – this will produce a warning if you made changes.
:q!	Quit the file and discard changes
:w filename	Save to filename
:w! filename	Overwrites to filename
:wq	Save and exit the file
:colorscheme[space][tab]	Get a selectable list of colour themes
:color <theme>	Directly select a colour theme

- 2.2.1 Open up the file in `vi` and delete the header of the file `HinfKW20_features_copy.txt`. Save it and compare file sizes with the original.

```
vi HinfKW20_features_copy.gtf
Move around the file with arrow keys, ctrl and shift
i                               # Enter edit mode
Remove some of the first comment line with [backspace]
[Esc]                          # Exit editing mode
dd                              # do multiple times to delete the first 3 # lines
:wq                            # Save changes and quit vi
ll                             # What sizes are our gtf files now?
```



```
mokca@MokData:~/FGDS/Module4$ ls -la
total 8640
drwxr-xr-x 2 mokca mokca 4096 Nov 18 13:36 .
drwxrwxrwx 6 mokca mokca 4096 Nov 18 12:45 ..
-rwxr-xr-x 1 mokca mokca 1875338 Nov 18 12:44 HinfKW20_cds.fna
-rwxr-xr-x 1 mokca mokca 2219091 Nov 18 13:28 HinfKW20_features.gtf
-rwxr-xr-x 1 mokca mokca 2218994 Nov 18 13:36 HinfKW20_features_copy.gtf
-rwxr-xr-x 1 mokca mokca 1853084 Nov 18 12:44 HinfKW20_genomic.fna
-rwxr-xr-x 1 mokca mokca 665876 Nov 18 12:44 HinfKW20_protein.faa
```

2.3.0 Concatenate files directly with `cat`

The `cat` command is useful when working with multiple files (like sequencing data) that you'd like to combine into a single file. Likewise, header-less data sets can be combined without opening an editor, copying, pasting, saving, etc.

- 2.3.1 Concatenate the `HinfKW20_cds.fna` and `HinfKW20_genomic.fna` files together to get all those DNA sequences into a single file.

```
ll *.fna
cat HinfKW20_cds.fna HinfKW20_genomic.fna > HinfKW20_allSeq.fna
# remember > means you'll be sending output to a file (not a program)

ll *.fna
```

```

mokca@MokData:~/FGDS/Module4$ ll *.fna
-rwxr-xr-x 1 mokca mokca 1875338 Nov 18 12:44 HinfKW20_cds.fna*
-rwxr-xr-x 1 mokca mokca 1853084 Nov 18 12:44 HinfKW20_genomic.fna*
mokca@MokData:~/FGDS/Module4$ cat HinfKW20_cds.fna HinfKW20_genomic.fna > HinfKW20_allSeq.fna
mokca@MokData:~/FGDS/Module4$ ls -la *.fna
-rw-r--r-- 1 mokca mokca 3728422 Nov 18 13:38 HinfKW20_allSeq.fna
-rwxr-xr-x 1 mokca mokca 1875338 Nov 18 12:44 HinfKW20_cds.fna
-rwxr-xr-x 1 mokca mokca 1853084 Nov 18 12:44 HinfKW20_genomic.fna

```

2.4.0 Use `grep` to search through a file

We spent some time in Module 1 going over regular expressions when editing text. We saw how powerful it could be to use this language to search through our text files. `grep` is an essential command-line utility which has its origins in Unix but is now available for all Unix-like systems and exists in variant forms even in the Windows environment and many programming languages. `grep` (`ggrep` in MacOS) generally takes the form of:

`grep -switches "searchPattern" filePath`

Switches	Description
-E	Use full regular expressions as defined in the bash shell
-P	Use Perl-compatible regular expressions
-i	Match without regard to case
-v	Invert the search
-c	Count the results in the file
-l	List only file names containing matches
-h	Hide filenames in the output
-n	Append file line numbers to the output

- 2.4.1 Count all the unique lines containing occurrences of tRNA and rRNA from the **HinfKW20_features_copy.gtf** file using the `grep` (`ggrep` in MacOS) command and in doing so, explore the functionality of the `>` and `>>` redirectors.

```

ls -l *.gtf
grep -ci "tRNA" HinfKW20_features_copy.gtf
grep -ci "rRNA" HinfKW20_features_copy.gtf

```

```

mokca@MokData:~/FGDS/Module4$ ls -l *.gtf
-rwxr-xr-x 1 mokca mokca 2219091 Nov 18 13:28 HinfKW20_features.gtf
-rwxr-xr-x 1 mokca mokca 2218994 Nov 18 13:36 HinfKW20_features_copy.gtf
mokca@MokData:~/FGDS/Module4$ grep -ci "tRNA" HinfKW20_features_copy.gtf
147
mokca@MokData:~/FGDS/Module4$ grep -ci "rRNA" HinfKW20_features_copy.gtf
60

```

- 2.4.2 Redirect the output of `grep` to a file using the `>` redirectors. Notice how the output of the `grep` command looks? The output highlights in red all occurrences of our search pattern, even if they should appear on multiple lines! Are we retrieving only **tRNA** features?

```

grep -in "tRNA" HinfKW20_features_copy.gtf # Note the output!
grep -i "tRNA" HinfKW20_features_copy.gtf > HinfKW20_tRNA_features.txt
ls -l *.txt

```

```

mokca@MokData:~/FGDS/Module4$ grep -i "tRNA" HinfKW20_features_copy.gtf > HinfKW20_tRNA_features.txt
mokca@MokData:~/FGDS/Module4$ ls -l *.txt
-rw-r--r-- 1 mokca mokca 46481 Nov 18 13:44 HinfKW20_tRNA_features.txt
mokca@MokData:~/FGDS/Module4$

```

2.4.3 Append the output of **grep** to a file using the **>>** redirectors. Rather than overwriting our destination file, we can append to our **HinfKW20_tRNA_features.txt** using the correct redirection symbol. Then we'll rename the file with the **mv** command.

```
grep -i "rRNA" HinfKW20_features_copy.gtf >> HinfKW20_tRNA_features.txt
mv HinfKW20_tRNA_features.txt HinfKW20_RNA_features.txt
```

```
mokca@MokData:~/FGDS/Module4$ grep -i "rRNA" HinfKW20_features_copy.gtf >> HinfKW20_tRNA_features.txt
mokca@MokData:~/FGDS/Module4$ mv HinfKW20_tRNA_features.txt HinfKW20_RNA_features.txt
mokca@MokData:~/FGDS/Module4$ less HinfKW20_RNA_features.txt
mokca@MokData:~/FGDS/Module4$ ls -l *.txt
-rw-r--r-- 1 mokca mokca 58562 Nov 18 13:49 HinfKW20_RNA_features.txt
```

```
less HinfKW20_RNA_features.txt          # How many lines do we have?
ls -l *.txt
```

2.4.4 How do we ensure we are counting only tRNA or rRNA features from our file? Based on our earlier **grep** output, it appears that feature types will include a “gbkey” entry in the final column that takes the format:

gbkey "tRNA" or **gbkey "rRNA"**

With that information, we can turn to Perl-compatible regular expressions with the **-P** switch. Remember what we learned earlier in **Module 1**?

```
grep -Pci "gbkey \"[r,t]RNA\" HinfKW20_features_copy.gtf
```

Is there a difference between our latest search and the RNA-based features file we created?

Stick with the -P! By using PCRE we can use meta characters like tabs (**\t**) and spaces (**\s**)! There are [other methods for utilizing tabs](#) as well but they can be a little more complex. Using PCRE opens up additional features not provided by basic regular expressions (default) or even extended (-E) regular expressions.

2.5.0 Count words and lines with **wc**

Whenever you want to quickly investigate a file, the output of a command, or track how a file might be updating from another process, you can use the **wc** (**w**ord **c**ount) command to check the number of words, lines, or characters. This command generally takes the form of:

wc -switches filePath

Switches	Description
-l	Count the number of lines
-w	Count the number of words
-m	Count the number of characters

2.5.1 Count the number of words and lines in **HinfKW20_cds.fna** using the **wc** command.

```
less HinfKW20_cds.fna          # Take a quick look at the file
-N
G                               # What is the last line

wc HinfKW20_cds.fna           # This will output lines, words, characters
```

2.5.2 Count and compare the number of features in the **HinfKW20_cds.fna**, the **HinfKW20_genomic.fna**, and the **HinfKW20_protein.faa** files by combining the **grep** (**ggrep** in MacOS) command and the **wc** command using **|**. The pipe symbol (**|**) redirects output from one command to the next through what is known as “standard input”, taking the place of explicit input (ie **filePath** in this case).

Note that all fasta file entry headers start with the > symbol and that that **grep -c ">" filename** would also have worked here, but we wouldn't have gotten to see how **|** can be used to combine commands.

```
less HinfKW20_cds.fna
grep ">" HinfKW20_cds.fna
grep ">" HinfKW20_cds.fna | wc
grep ">" HinfKW20_cds.fna | wc -l
grep ">" HinfKW20_genomic.fna | wc -l
grep ">" HinfKW20_protein.faa | wc -l
```

```
mokca@MokData:~/FGDS/Module4$ grep ">" HinfKW20_cds.fna | wc
1709 14998 286544
mokca@MokData:~/FGDS/Module4$ grep ">" HinfKW20_cds.fna | wc -l
1709
mokca@MokData:~/FGDS/Module4$ grep ">" HinfKW20_genomic.fna | wc -l
1
mokca@MokData:~/FGDS/Module4$ grep ">" HinfKW20_protein.faa | wc -l
1709
```

2.6.0 Quickly extract from tabular data with **cut**

When you generate data that might be in a tabular format, you may wish to look at specific columns or change the delimiter of the file (csv versus tsv, etc.). This may be after a series of commands or as part of a series where you wish to reformat or isolate output on the fly. The **cut** (**gcut** in MacOS) command takes the form of:

cut -switches parameters filePath (Windows)

gcut -switches parameters filePath (MacOS)

Key	Description
-f, --fields=LIST	Specify columns to return (comma-separated list)
-d, --delimiter=DELIM	Specify delimiter used to read/identify columns (tab is default)
-c, --characters=LIST	Specify range of characters to use (don't use on tabular data!)
--output-delimiter=STRING	Specify the type of delimiter to use for resulting output
--complement	Complement the set of selected fields, or characters

2.6.1 Generate a simplified gene features file that only contains essential information regarding where and what each CDS feature is using the **cut** command (**gcut** in MacOS). Call this new file **HinfKW20_features_long.txt**.

```
ls -l *.gtf
grep -Pi "\tCDS" HinfKW20_features_copy.gtf | wc -l
grep -Pi "\tCDS" HinfKW20_features_copy.gtf | cut -f 1,4,5,7,9
grep -Pi "\tCDS" HinfKW20_features_copy.gtf | cut -f 1,4,5,7,9 >
HinfKW20_features_long.txt

less HinfKW20_features_long.txt
ls -l *.txt
```

```
L42023.1      2      1018  +      gene_id "HI_0001"; transcript_id "unassigned_transcript_1"; gbkey "CDS"; lo
L42023.1     1190     3010  +      gene_id "HI_0002"; transcript_id "unassigned_transcript_2"; gbkey "CDS"; lo
L42023.1     3053     3838  -      gene_id "HI_0003"; transcript_id "unassigned_transcript_3"; gbkey "CDS"; lo
L42023.1     3857     4318  -      gene_id "HI_0004"; transcript_id "unassigned_transcript_4"; gbkey "CDS"; lo
L42023.1     4582     5391  -      gene_id "HI_0005"; transcript_id "unassigned_transcript_5"; gbkey "CDS"; lo
L42023.1     8750     9685  +      gene_id "HI_0007"; transcript_id "unassigned_transcript_6"; gbkey "CDS"; lo
L42023.1     9681    10394  +      gene_id "HI_0008"; transcript_id "unassigned_transcript_7"; gbkey "CDS"; lo
L42023.1    10467    11372  +      gene_id "HI_0009"; transcript_id "unassigned_transcript_8"; gbkey "CDS"; lo
```

2.7.0 Combine columns with paste

Looking at our output from above, we can see that the 5th column we generated is actually a semi-colon-separated list of additional information. Some key pieces of information in this last column are actually the **gene_id**, **product** and **protein_id** entries in the 1st/8th/9th position of this list. How can we extract this information into its own columns? We'll produce two sets of **cut** (**gcut** in MacOS) data and then proceed to combine their columns with the **paste** function. The paste command takes the form of:

```
paste -switches input1 input2 ... inputN
```

Key	Description
-d, --delimiters=LIST	Specify delimiter(s) used to paste columns together (tab is default). Note the delimiter must come immediately after the switch!
-s, --serial	Paste one file at a time instead of in parallel

Two last notes for new command-line syntax that we'll make use of:

- 1) The `-` character can be substituted as a placeholder for the standard input. This is a powerful way to place our incoming input from a pipeline into the correct position for a call.
- 2) When working with in the commandline, we use `$'...'` quoting so we can substitute for certain escaped strings. With this syntax they are interpreted directly instead of quoted literally. In the case of a `"\t"` we want it to be interpreted as a tab-space! Note that you **MUST** use single-quotes for this to work correctly.

```
cut HinfKW20_features_long.txt -f 5
cut HinfKW20_features_long.txt -f 5 | cut -d";" -f 1,8,9 --output-delimiter
    $'\t'
cut HinfKW20_features_long.txt -f 5 | cut -d";" -f 1,8,9 --output-delimiter
    $'\t' | paste HinfKW20_features_long.txt -
cut HinfKW20_features_long.txt -f 5 | cut -d";" -f 1,8,9 --output-delimiter
    $'\t' | paste HinfKW20_features_long.txt - | cut -f 1,2,3,4,6,7,8
cut HinfKW20_features_long.txt -f 5 | cut -d";" -f 1,8,9 --output-delimiter
    $'\t' | paste HinfKW20_features_long.txt - | cut -f 1,2,3,4,6,7,8 >
    HinfKW20_features_simple.txt
less HinfKW20_features_simple.txt
```

Our final command can be broken down into 5 segments:

cut HinfKW20_features_long.txt -f 5	Cut out the 5 th column of HinfKW20_features_long.txt
cut -d";" -f 1,8,9 --output-delimiter \$'\t'	Cut that by ";" and take cols 1,8,9 and separate by <code>\t</code>
paste HinfKW20_features_long.txt -	Combine the original file with the 3 new columns
cut -f 1,2,3,4,6,7,8	Cut out the 5 th column
> HinfKW20_features_simple.txt	Save the output to HinfKW20_features_simple.txt

2.8.0 Sort your data with `sort`

Looking at our output from the last section, we see that it's all sorted by ascending position. Suppose you want to quickly `sort` your tabular data in ascending or descending order. Again, this could be pre-, post- or mid-pipeline. The `sort` command (`gsort` in MacOS) takes the form of:

```
sort -switches parameters filePath
```

Switches	Description
-k	Specify specific column number to sort by
-t	Specify delimiter to separate columns (normally delimits by blank spaces)
-n	Sort by numerical string value
-r	Sort in reverse order

2.8.1 Sort the output **HinfKW20_features_simple.txt** file by feature name rather than by their location in the genome using the `sort` command.

```
ll *.txt
>
HinfKW20_features_simple_sortedbyproduct.txt
less HinfKW20_features_simple_sortedbyproduct.txt
ls -l *.txt
```

```
L42023.1      1126894 1127337 -      gene_id "HI_1062"      product "(3R)-hydroxymyristol (acyl carrier protein) c
L42023.1      1436783 1438972 +      gene_id "HI_1357"      product "1,4-alpha-glucan branching enzyme (glgB) "
L42023.1      525214 526137 -      gene_id "HI_0509"      product "1,4-dihydroxy-2-naphthoate octaprenyltransfe
L42023.1      788250 788969 -      gene_id "HI_0734"      product "1-acyl-glycerol-3-phosphate acyltransferase
L42023.1      1527362 1529236 +      gene_id "HI_1439"      product "1-deoxyxylulose-5-phosphate synthase (dxs) (E
L42023.1      469940 470878 -      gene_id "HI_0447"      product "1-phosphofructokinase (fruK)" protein_id "A
L42023.1      1647859 1648323 -      gene_id "HI_1579"      product "15 kDa peptidoglycan-associated lipoprotein
L42023.1      1316432 1317127 -      gene_id "HI_1243"      product "16s pseudouridylate 516 synthase (rsuA) "
```

2.8.2 Sort the output **HinfKW20_features_simple.txt** file by column 3 using the `sort` command using both the string and numerical string values.

```
sort -k 3 HinfKW20_features_simple.txt | less
```

You may notice that there is a big difference between our two kinds of `sort` outputs. In the one instance, it's more of an alphabetical sort, not accounting for the context of non-leading 0 numbers. In the latter case, the `-n` switch attempts to give context to the column values by assuming these are numerical values!

3.0.0 Creating bash scripts and regulating access to files or programs

We are now going to learn how to combine commands into some simple scripts and how to regulate file access. This is where understanding your file system and directory tree really comes into play. Just like when you are in a Finder or File Explorer window, you can't open a file or run a program that is not in your current folder, but there are ways to manage this effectively. To begin, let's review some more helpful commands and files that are part of your Linux OS.

Command	Description
bash	Execution command (e.g. <code>bash script.sh</code>).
sudo	Super user permissions for system files (e.g. <code>sudo rm systemfile.txt</code>).
\$PATH	Directories whose files or programs can be accessed from anywhere on your system.
.bash_profile/ .bashrc	Configuration script file in your root directory that is run every time you enter the shell. It is used to modify your \$PATH (<code>.bash_profile</code> or <code>.zshrc</code> for MAC and <code>.bashrc</code> for LINUX).

3.1.0 Alter file permissions with `chmod`

When working with files, there are three general tasks you'd like to do with your files: read, write, and execute (if they are programs). In order to use any of these options, a file must have those specific permissions enabled. Furthermore, every file keeps track of the general types of people (levels) that can access it: you (user), groups who own/share the file with you (group), and world-wide access or other users on the system (other).

When using the `chmod` command, it takes the general form of

```
chmod options level[mode]permission, ... filePath
```

or

```
chmod options 3-digit-octal filePath
```

Switches/Syntax	Description
+, -, =	MODE: Add, subtract, or set directly the permissions of a level
u, g, o, a	LEVEL tags for user, group, other, or all
r, w, x	PERMISSION tags to set for read, write, and execute
4, 2, 1	The OCTAL values for read, write, and execute. These can be added to make specific combinations of permissions ie 6 -> read and write, vs 7 -> read, write, and execute and are set in a three-digit value to represent user/group/other
-v, --verbose	Output a diagnostic for every file processed
-R, --recursive	Change files and directories recursively

Let's try some examples and see how it works in more detail.

3.1.1 `chmod` the permissions of all txt files to read for all levels and write only for the user and group.

```
ll *.txt
chmod u=rw,g=rw,o=r *.txt
ll *.txt
```

3.1.2 `chmod` the permissions of all txt files with octal to read and write for user and group levels only and no access for other.

```
chmod 660 *.txt
ll *.txt
```

```

mokca@MokData:~/FGDS/Module4$ ll *.txt
-rw-r--r-- 1 mokca mokca 58562 Nov 18 13:49 HinfKW20_RNA_features.txt
-rw-r--r-- 1 mokca mokca 600802 Nov 18 14:05 HinfKW20_features_long.txt
-rw-r--r-- 1 mokca mokca 197020 Nov 18 14:13 HinfKW20_features_simple.txt
-rw-r--r-- 1 mokca mokca 197020 Nov 18 14:17 HinfKW20_features_simple_sortedbyproduct.txt
mokca@MokData:~/FGDS/Module4$ chmod u=rw,g=rw,o=r *.txt
mokca@MokData:~/FGDS/Module4$ ll *.txt
-rw-rw-r-- 1 mokca mokca 58562 Nov 18 13:49 HinfKW20_RNA_features.txt
-rw-rw-r-- 1 mokca mokca 600802 Nov 18 14:05 HinfKW20_features_long.txt
-rw-rw-r-- 1 mokca mokca 197020 Nov 18 14:13 HinfKW20_features_simple.txt
-rw-rw-r-- 1 mokca mokca 197020 Nov 18 14:17 HinfKW20_features_simple_sortedbyproduct.txt
mokca@MokData:~/FGDS/Module4$ chmod 660 *.txt
mokca@MokData:~/FGDS/Module4$ ll *.txt
-rw-rw---- 1 mokca mokca 58562 Nov 18 13:49 HinfKW20_RNA_features.txt
-rw-rw---- 1 mokca mokca 600802 Nov 18 14:05 HinfKW20_features_long.txt
-rw-rw---- 1 mokca mokca 197020 Nov 18 14:13 HinfKW20_features_simple.txt
-rw-rw---- 1 mokca mokca 197020 Nov 18 14:17 HinfKW20_features_simple_sortedbyproduct.txt

```

3.2.0 Generating your first bash script

A bash script is a plain text file that carries a series of commands that can be executed by the command-line. All the calls we have used can be included in a bash script to run and manipulate our files. Any commands from your bash script can be run directly in the command-line interface. We combine these commands into a single bash script to save time and “automate” our commands.

- 3.2.1 Create a new file titled **bash_script.sh** in a new directory titled **scripts**. This directory should be a subdirectory in the **Module4** directory. **.sh** is an extension that is commonly applied to executable bash scripts. Make this file executable for all users with **chmod**. Note how the permissions on the file have changed.

```

mkdir scripts
cd scripts
vi bash_script.sh      # alternatively "touch bash_script" or
                       # "> bash_script"
:wq
ls -la

```

```

mokca@MokData:~/FGDS/Module4$ mkdir scripts
mokca@MokData:~/FGDS/Module4$ cd scripts
mokca@MokData:~/FGDS/Module4/scripts$ vi bash_script.sh
mokca@MokData:~/FGDS/Module4/scripts$ ls -la
total 8
drwxr-xr-x 2 mokca mokca 4096 Nov 18 15:17 .
drwxr-xr-x 3 mokca mokca 4096 Nov 18 15:17 ..
-rw-r--r-- 1 mokca mokca  0 Nov 18 15:17 bash_script.sh
mokca@MokData:~/FGDS/Module4/scripts$ chmod a+rw bash_script.sh
mokca@MokData:~/FGDS/Module4/scripts$ ls -l
total 0
-rwxrwxrwx 1 mokca mokca 0 Nov 18 15:17 bash_script.sh

```

3.3.0 Edit your bash script

Now that we have our bash script file, we can edit it again with **vi** to add some commands. We’re going to include the following text in our bash script which will count the number of files in a directory and then count the number of lines, words, and characters in each file.

This script can then be run using the **bash** command. Recall that the **vi** command gets you into the script, the **i** key allows you to insert text into the file, the **esc** key cancels insert mode, and the **:wq** command sequence allows you to **w**rite (save) the script and **q**uit.

In many programming languages, everything to the right of a **#** (pound) sign within a script file is ignored (not interpreted nor executed). This feature allows us to add comments to our code to make it more readable and easier to understand, without introducing bugs in our code

```
#!/bin/bash # this is called a shebang. Tells the system that this is a UNIX
executable file.

echo "This is how many files are in your directory:"
ls | wc -l # list files and pipes its output to count lines

echo "This is the number of lines, words, and characters in each file:"
# echo prints text into the console

for file in * # this is something called a for loop
# It will create a list of each file in the current directory
do
    wc -l $file # The $ refers to each file detected by out for loop
    wc -w $file
    wc -m $file
done
```

3.3.1 Open up vi, edit the file, insert the above code and save the file.

```
vi bash_script.sh
i
# copy and paste the above code into vi
<esc> # escape insert (writing mode)
:wq # save (write) changes and quit
less bash_script.sh
# this should run even for zsh terminals!
```

```
mokca@MokData:~/FGDS/Module4/scripts$ ls -la
total 12
drwxr-xr-x 2 mokca mokca 4096 Nov 18 15:22 .
drwxr-xr-x 3 mokca mokca 4096 Nov 18 15:17 ..
-rwxrwxrwx 1 mokca mokca 469 Nov 18 15:22 bash_script.sh
mokca@MokData:~/FGDS/Module4/scripts$ bash bash_script.sh
This is how many files are in your directory:
1
This is the number of lines, words, and characters in each file:
15 bash_script.sh
97 bash_script.sh
469 bash_script.sh
```

3.4.0 Run your bash script from other directories

Now try running the script from your **Module4** directory. This won't work unless you specify the absolute or relative path to your script. When you do specify the relative path, the script will run on your current directory. However, it won't be able to give you file sizes for any sub-directories like **scripts**.

3.4.1 Run your bash script from a different directory. You'll want to properly locate it by absolute or relative location.

```
cd ..
bash bash_script.sh # Where do we keep the script?
bash scripts/bash_script.sh
```

```

mokca@MokData:~/FGDS/Module4$ bash bash_script.sh
bash: bash_script.sh: No such file or directory
mokca@MokData:~/FGDS/Module4$ bash scripts/bash_script.sh
This is how many files are in your directory:
11
This is the number of lines, words, and characters in each file:
207 HinfKW20_RNA_features.txt
6497 HinfKW20_RNA_features.txt
58562 HinfKW20_RNA_features.txt
45023 HinfKW20_allSeq.fna
58319 HinfKW20_allSeq.fna
3728422 HinfKW20_allSeq.fna
22145 HinfKW20_cds.fna
35434 HinfKW20_cds.fna
1875338 HinfKW20_cds.fna
6978 HinfKW20_features.gtf
246620 HinfKW20_features.gtf
2219091 HinfKW20_features.gtf
6975 HinfKW20_features_copy.gtf
246614 HinfKW20_features_copy.gtf
2218994 HinfKW20_features_copy.gtf
1709 HinfKW20_features_long.txt
62669 HinfKW20_features_long.txt
600802 HinfKW20_features_long.txt
1709 HinfKW20_features_simple.txt
21834 HinfKW20_features_simple.txt
197020 HinfKW20_features_simple.txt
1709 HinfKW20_features_simple_sortedbyproduct.txt
21834 HinfKW20_features_simple_sortedbyproduct.txt
197020 HinfKW20_features_simple_sortedbyproduct.txt
22878 HinfKW20_genomic.fna
22885 HinfKW20_genomic.fna
1853084 HinfKW20_genomic.fna
9058 HinfKW20_protein.faa
22347 HinfKW20_protein.faa
665876 HinfKW20_protein.faa
wc: scripts: Is a directory
0 scripts
wc: scripts: Is a directory
0 scripts
wc: scripts: Is a directory
0 scripts

```

3.5.0 Add scripts to your `$PATH` variable to access them anywhere

As you can see from our above example, when using the bash script we generated, it is important to know where the script is located. However, navigating to the location of a script when you are in a completely different directory can make your commands quite long. You can get around this by storing scripts close to the root or home directory OR you can make them part of your `$PATH` variable.

The `$PATH` environmental variable is an **ordered** list of paths that Linux will search through for executables when running a command. You do not need to include the executable itself but just the directory where it is located.

- 3.5.1 Add the scripts directory to your `$PATH` so that any script in the directory **scripts** can be run from anywhere on your system without having to specify the absolute or relative path to the script. You will need to edit your `$PATH` through your **.bash_profile** or **.zshrc** (Mac OS) or **.bashrc** (Windows/linux) to accomplish this.

```

cd ~
echo $PATH # See what your current $PATH setup includes
vi .bashrc # macOS: vi .bash_profile or .zshrc
# Scroll to the bottom of the file
i
# Directories to add to path
PATH="$PATH:
<esc>
:wq

```

3.5.2 Source your `.bash_profile`, `.zshrc` or `.bashrc` file and confirm it has been changed. By using `source`, you are “re-initializing” your current command-line session.

```

source .bashrc # macOS: source .bash_profile or .zshrc
echo $PATH
cd FGDS/Module4
bash bash_script.sh

```

```

mokca@MokData:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Common:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA NvDLISR:/mnt/c/Users/mokca/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/mokca/AppData/Local/GitHubDesktop/bin:/snap/bin:/home/mokca/FGDS/Module4/scripts
mokca@MokData:~$ cd FGDS/Module4
mokca@MokData:~/FGDS/Module4$ bash bash_script.sh
This is how many files are in your directory:
11
This is the number of lines, words, and characters in each file:
207 HinfKW20_RNA_features.txt
6497 HinfKW20_RNA_features.txt
58562 HinfKW20_RNA_features.txt
45023 HinfKW20_allSeq.fna
58319 HinfKW20_allSeq.fna
3728422 HinfKW20_allSeq.fna

```

When you use CTRL + C instead
of copying using right click



4.0.0 Running operations on a remote server

The last thing we are going to learn today is how to access data and perform operations on a remote server. Genomic data and analyses often occur on remote servers that have more storage and computing power than your personal computers. Here are some important commands for running operations on remote servers:

Switches/Syntax	Description
ssh	Start a secure remote shell connection (e.g. <code>ssh fgds@142.150.215.186 -p 24</code>)
-p	Specify the port to use
sftp	Start a file transfer remote shell connection (e.g. <code>sftp -oPort=24 fgds@142.150.215.186</code>)
get	Transfer from the server to your computer (e.g. <code>get HinfKW20_cds.fna</code>)
put	Transfer from your computer to the server (e.g. <code>put HinfKW20_cds.fna</code>)
l	Place before any command to run it on your computer rather than the server (e.g. <code>lcd</code>)
quit	Close the sftp connection
scp	Securely copy files to/from remote location (e.g. <code>scp HinfKW20_cds.fna fgds@142.150.214.76/home/HinfKW20_cds.fna</code>)

4.1.0 Connect and explore with `ssh`

We can use the secure shell protocol to communicate between two computers. This is more commonly known as `ssh` and uses an encrypted connection that is suitable for use even on insecure networks. We'll briefly explore the basics of using `ssh` to connect, alter and explore another system.

- 4.1.1 Connect to our course server through an `ssh` connection and create a new directory for yourself with an empty file using `mkdir` and `vi`. Make sure to replace the directory name with **your own name** after "cagef_fgds". You will always disconnect from a server using the `exit` command.

```
ssh fgds@142.150.215.186
    PW = fall2025
mkdir cagef_fgds_Your_name
cd cagef_fgds_Your_name
touch myemptyfile.txt          # Use touch to quickly make a file
ls -l
exit
```

```
mokca@MokData:~/FGDS/Module4$ ssh fgds@142.150.215.186
fgds@142.150.215.186's password:
Last login: Mon Nov 18 15:53:00 2024 from 99.241.85.1
fgds@esc4037-darwin-guttman:~$ mkdir cagef_fgds_calvinmok
fgds@esc4037-darwin-guttman:~$ cd cagef_fgds_calvinmok/
fgds@esc4037-darwin-guttman:~/cagef_fgds_calvinmok$ touch myemptyfile.txt
fgds@esc4037-darwin-guttman:~/cagef_fgds_calvinmok$ ls -l
total 0
-rw-r--r--. 1 fgds fgds 0 Nov 18 15:55 myemptyfile.txt
fgds@esc4037-darwin-guttman:~/cagef_fgds_calvinmok$ exit
logout
Connection to 142.150.215.186 closed.
```

4.2.0 Send and retrieve files with `sftp`

The `sftp` or SSH File Transfer Protocol (aka Secure File Transfer Protocol) is another network protocol that uses `ssh` to facilitate file access, transfer, and management. This protocol uses the user-authenticated credentials required for an SSH connection.

- 4.2.1 Connect to our course server through a `sftp` connection and exchange content using the `get` and `put` commands. Note that all basic commands will apply to the server, but you can interact with your local computer by placing a `l` before the command when you're in a `sftp` connection.

```
cd cagef_fgds_YourName # replace with your own name
ls
get myemptyfile.txt      # "get" downloads myemptyfile.txt
lls # the preceding "l" points to your local computer (e.g. your laptop)
put HinfKW20_protein.faa # upload HinfKW20_protein.faa to the server
ls
exit
```

That's it (for today)!



5.0.0 Class summary

That concludes our fourth lecture and introduction to the command-line tools and file structure. Next module we will take a closer look at installing programs in various ways through the command-line and contrast how some methods may be more appropriate to different needs of the data scientist. Altogether we've explored the following in this module:

- File and directory creation.
- Altering file content and file permissions.
- Running basic command-line tools for viewing and summarizing files.
- Creating your first bash script
- Setting up your `$PATH` variables
- Connecting to remote systems with `ssh` and `sftp`

5.1.0 Post-lecture assessment (9% of final grade)

Soon after this lecture, a homework assignment will be made available on Quercus in the assignment section. It will build on the ideas and/or data generated within this lecture. Each homework assignment will be worth 9% of your final mark. If you have assignment-related questions, please try the following steps in the order presented:

- Check the internet for a solution – read forums and learn to navigate for answers.
- Generate a discussion on Quercus outlining what you've tried so far and see if other students can contribute to a solution.
- Contact course teaching assistants or the instructor.

5.2.0 Suggested class preparation for Module 5

Next week we will begin exploring installations in the command-line interface and learning navigate our way through the spaghetti-string connections that result. There is nothing to help prepare for this except raw experience and lots of googling.

Next week we will need to install a package named **Anaconda**. This will make your life much easier BUT it will require about 30GB to install all of its components and an additional 15GB to store data for the rest of the course. Overall ensure you have ~50GB of free space on your hard drive.

5.3.0 Acknowledgements

This course was originally conceived and produced by Dr. David Guttman and Dr. Marcus Dillon.

- **Revision 1.0.0:** edited and prepared by David Guttman, Ph.D. and Marcus Dillon, Ph.D.
- **Revision 2.0.0:** edited and prepared for *CSB1021HF LEC0131, 10-2021* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.
- **Revision 2.0.1:** edited and prepared for *CSB1021HF LEC0131, 10-2023* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.
- **Revision 2.0.2:** edited and prepared for *CSB1021HF LEC0131, 10-2024* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.
- **Revision 2.0.3:** edited and prepared for *CSB1021HF LEC0131, 10-2025* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.

Appendix 1: Shell (Terminal) Commands

From Haddock & Dunn. 2011. Practical Computing for Biologists. Sinauer Assoc. Sunderland MA

Terminal operations are described in Chapters 4–6, 16, and 20. Many of the built-in bash shell commands are summarized here for quick reference. To get more information about a command and its options, type `man`, followed by the name of the command. If you are not sure which command applies, you can also search the contents of the help files using `man -k` followed by a keyword term.

Command	Description	Usage
ls	List the files in a directory	<code>ls -la</code>
	Parameters that follow can be folder names (use * as a wildcard)	<code>ls -l *.txt</code>
		<code>ls -FG scripts</code>
		<code>ls ~/Documents</code>
	<code>-a</code> Show hidden files	<code>ls /etc</code>
	<code>-l</code> Show dates and permissions	
	<code>-1</code> List the file names on separate lines. Useful as a starting point for regex into a list of commands	
cd	<code>-G</code> Enable color-coding of file types	
	<code>-F</code> Show a slash after directory names	
	Change directory	
	Without a slash, names are relative to the current directory	<code>cd scripts</code>
	With a preceding slash (/) names start at the root level	<code>cd /User</code>
	Tilde (~/) starts at the user's home directory	<code>cd ~/scripts</code>
	Two dots (..) goes "up" to the enclosing directory	<code>cd My\ Documents</code>
	One dot refers to the current directory	<code>cd 'My Documents'</code>
	Minus sign goes to the previously occupied directory	<code>cd ../..</code>
	Use <code>tab</code> key (see below) to auto-complete partially typed paths	<code>cd ..</code>
	Use backslash before spaces or strange characters in the directory name, or put the whole name in quotes	<code>cd -</code>

Command	Description	Usage
pwd	Print the working directory (the path to the folder you are in)	
	 key to step back through previously typed commands The cursor can be repositioned with the  and  keys, and commands can then be edited Press  from anywhere in the line to re-execute. On OS X you can also reposition by -clicking at a cursor location	
	Auto-complete file, folder, or script names at the command line	cd ~/Doc 
less	Show contents of a file, page by page These commands also apply to viewing the results of man While less is running: q Quit viewing  Next page b Back a page 15 g Go to line 15 G Go to the end  or  Move up or down a line /abc Search file for text abc n After an initial search, find next occurrence of the search item ? Find previous occurrence of the search item h Show help for less	less data.txt
mkdir	Make a new directory (a new folder)	mkdir scripts
rmdir	Remove a directory (folder must be empty)	rmdir ~/scripts
rm	Remove file or files Use the -f flag to delete without confirmation (careful!) Use the -r flag to recursively delete the files in a directory and then the directory itself	rm test.txt rm -f *_temp.dat
man	Show the manual pages for a Unix command Use -k to search for a term within all the manuals The result is displayed using the less command above, so the same shortcuts allow you to navigate through	man mkdir man -k date man chmod

Command	Description	Usage
cp	Copy file, leaving original intact Does not work on folders themselves Single period as destination copies file to current directory, using same name	cp test1.txt test1.dat cp temp ../temp cp ../test.py .
mv	Move file or folder, renaming or relocating it Unlike cp, this does work on directories	mv test1.txt test1.dat mv temp ../temp2
	Pipe output of one command to the input of another command	history grep lucy
>	Send output of a command to a file, overwriting existing files Do not use a destination file that matches a wildcard on the left side	ls -l *.py > files.txt
>>	Send output of a command to a file, appending to existing files	echo "#Last line" >> data.txt
<	Send contents of a file into command that supports its contents as input	mysql -u root midwater < data.sql
./	Represents the current directory in a path—the same location as pwd Trailing slash is optional Can execute a file in the current directory even when the file directory is not included in the PATH	cp ../*.txt ./ ./myscript.py
cat	Concatenate (join together) files without any breaks. Streams the contents of the file list across the screen	cat README cat *.fta > fasta.txt
head	Show the first lines of a file or command Use the -n flag to specify the number of lines	head -n 3 *.fasta ls *.txt head
tail	Show the last lines of a file or output stream Use the -n flag to specify the number of lines to show With a plus sign, skip that number of lines and show to the end. Use -n +2 to show from the second line of the file to the end, skipping one header line	tail -n 20 *.fta tail -n +3 data.txt
wc	Count lines, words, and characters in an output stream or file	wc data.txt ls *.txt wc
which	Show the location of executable files in the system path	which man

Command	Description	Usage
grep	Search for phrase in a list of files or pipe and show matching lines: <pre>grep -E "searchterm" filelist</pre> Often used in conjunction with piped output: <code>command grep searchterm</code> Use quotes around search terms, especially spaces or punctuation like >, &, #, and others To search for tab characters, type <code>ctrl</code>V followed by <code>tab</code> inside the quotes Optional flags: -c Show only a count of the results in the file -v Invert the search and show only lines that do not match -i Match without regard to case -E Use full regular expressions Terms should be enclosed in quotes. Use <code>[]</code> to indicate a character range rather than the wildcards of Chapters 2 and 3 General wildcard equivalents: <pre>\s [[:space:]] \w [[:alpha:]] \d [[:digit:]]</pre> -l List only the filenames containing matches -n Show the line numbers of the match -h Hide the filenames in the output	
agrep	Search for approximate matches, allowing insertions, deletions, or mismatched characters. (Must be installed separately.) See Chapter 21 Optional flags include: -d ", " Use comma as delimiter between records -2 Return results with up to 2 mismatches. Maximum is 8 mismatches -B -y Return the best match without specifying a number of mismatches -l Only list file names containing matches -i Match without regard to case	<pre>agrep -d ">" -B -y ATG seqs.fta agrep -3 siphonafore taxa.txt</pre>
chmod	Change access permissions on a file (usually to make a script executable or Web accessible) First option is one of u, g, o for user, group, other Second option after the plus or minus is r, w, or x, for read, write, or execute. Can also use binary encoding as explained in Appendix 6	<pre>chmod u+x file.pl chmod 644 myfile.txt chmod 755 myscript.py</pre>

Command	Description	Usage
set	Show environmental variables, including functions that have been defined	
\$HOME	The environmental variable containing the path user's home directory	echo \$HOME cd \$HOME
\$PATH	The user's PATH variable, where the directories to search for commands are stored	export PATH=\$PATH:/usr/local/bin
nano	Invoke the text editor. Control key sequences include: <code>ctrl</code> X Exit nano (will be prompted to save) <code>ctrl</code> O Save file without exiting <code>ctrl</code> Y Scroll up a page <code>ctrl</code> V Scroll down a page <code>ctrl</code> C Cancel operation <code>ctrl</code> G Show help and list of commands	nano filename.txt
<code>ctrl</code> C	Interrupt the current process	
sort	Sort lines of a file -k N Sort using column number <i>N</i> instead of starting at the first character. Columns are delimited by a series of white space characters -t ", " In conjunction with -k, use commas as the delimiter to define columns -n Sort by numerical value instead of alphabetical -r Sort in reverse order -u Return only one unique representative from a series of identical sorted lines	sort -k 3 data.txt sort -k 2 -t ", " F1.csv sort -nr numbers.txt sort A.txt > A_sort.txt
uniq	Return a single line for each consecutive instance of that line in a file or output stream. To remove all duplicates from anywhere in the file, it must be sorted before being piped to the uniq command Use -c flag to return a count along with the repeated element	uniq -c records.txt sort names uniq -c

Command	Description	Usage
cut	Extract one or more columns of data from a file <code>-f 1,3</code> Return columns 1 and 3, delimited by tabs <code>-d ", "</code> Use commas as the field delimiter instead of tabs. Used in combination with <code>-f</code> <code>-c 3-8</code> Return characters 3 through 8 from the file or stream of data	<pre>cut -c 5-15 data.txt cut -f 1,6 data.csv cut -f2 -d ":" > Hr.txt</pre>
curl	Retrieve the contents of a URL from over the network. URL should be placed in quotes. Without additional parameters, will stream contents to the screen For some Linux versions, wget offers similar functionality See <code>man curl</code> for ways to send user login information at the same time <code>-o</code> Set the name of the output file to save individual files for the data. See #1 below <code>-m 30</code> Set a time out of 30 seconds <code>[01-25]</code> In the URL, substitute two digit numbers from 01 to 25 into the address in succession <code>{22,33}</code> Substitute items in brackets into URL <code>{A,C,E}</code> <code>#1</code> The substituted value, for use in generating the filename	<pre>curl "www.myloc.edu" > myloc.html curl "http://www.nasa.gov/weather[01-12]{1999,2000}" -m 30 -o weather#1_#2.dat</pre>
sudo	Run the command that follows as a superuser with privileges to write to system files	<pre>sudo python setup.py install sudo nano /etc/hosts</pre>
alias	Define a shortcut for use at the command line. To make persistent, add to startup settings file <code>.bash_profile</code> or equivalent	<pre>alias cx='chmod u+x'</pre>
function	Create a shell function—like a small script <code>\$1</code> is the first user argument supplied after the command is typed <code>\$@</code> is all the parameters—useful for loops as below Variable names are defined with the format <code>NAME=</code> with no spaces. They are retrieved with <code>\$NAME</code> Save it in <code>.bash_profile</code> to make it permanent	<pre>myfunction() { # insert commands here echo \$1 }</pre>
;	In a command or script, equivalent to pressing <code>return</code> and starting a new line	<pre>date; ls</pre>

Command	Description	Usage
for	Perform a for loop in the shell. Can be useful in the context of a function	for ITEM in *.txt; do echo \$ITEM done
if	An if statement in a shell function: <pre> if [test condition] then # insert commands else # alternate command fi </pre> Comparison operators are eq for equals, lt for less than and gt for greater than	if [\$# -lt 1] then echo "Less than" else echo "greater than 1" fi
` `	Backtick symbols surrounding a command cause the command to be executed and then substitute the output into that place in the shell command or script	cd `which python`/.. nano `which script.py`
host	Return IP number associated with a hostname, or the hostname associated with an IP address, if available	host www.sinauer.com host 127.0.0.1
ssh	Start a secure remote shell connection	ssh lucy@pcfb.org
scp	Securely copy files to or from a remote location	scp localfile user@host/path/remotefile scp user@host/home/file.txt localfile.txt
sftp	Start a file transfer connection to a remote site. The prompt changes to an ftp prompt, at which the following commands can be used: <pre> open From the prompt, open a new sftp connection get Bring a remote file to the local server put Place a local file on the remote system cd Change directory on the remote server lcd Change directory on the local machine quit Exit the sftp connection </pre>	sftp user@remotemachine
gzip gunzip zip unzip	Compress and uncompress files	gzip files.tar gunzip files.tar.gz zip archive.zip unzip archive.zip
tar	Create or expand an archive containing files or folders <pre> -cf Create -xvf Expand -xvzf Expand and uncompress gzip </pre>	tar -cf archive.tar ~/scripts tar -xvzf arch.tar.gz

Command	Description	Usage
&	When placed at the end of a command, runs it in the background	
ps	Show currently running processes. Flags controlling the output vary greatly by system. Usually a good starting point is -ax. See man ps for more	ps -ax grep lucy
top	Show current processes sorted by various parameters, most useful of which is processor usage -u	top -u
kill -9	Terminate a process emphatically, using its process ID. Retrieve PID from the ps or top command	kill -9 5567
killall	Terminate processes by name	killall Firefox
nohup	Run command in background and don't terminate it when logging out or closing the shell window Use in this odd format shown, to prevent program output to cause the command to quit	nohup <i>command</i> 2> /dev/null < /dev/null &
ctrl Z	Suspend the operation to move it into the background or perform other operations	
jobs	Show backgrounded or suspended jobs, won't show normal active processes	
bg	Move a suspended process into the background. Optional number after it in the format %1 will specify the job number	
apt-get yum rpm port	Package installers for various Unix distributions. Search for and install remote software packages. Typically used with sudo	sudo apt-get install agrep yum search imagemagick